VAJRA

Evidence-Driven Autonomous Cyber-Reasoning & Software Repair System

Detailed Technical Project Report

Version 1.0 • August 202c

A modular security engineering platform that discovers vulnerabilities, establishes evidence, generates minimal repairs, verifies them independently, learns from repair history, and produces an auditable repair-assurance record.

1. Executive Summary

VAJRA is proposed as an evidence-driven autonomous cyber-reasoning and software repair system. Its purpose is to move beyond conventional vulnerability scanners and beyond systems that simply ask a language model to write a patch. VAJRA combines multiple specialized security-analysis mechanisms—static analysis, dynamic analysis, fuzzing, dependency analysis, runtime evidence, and regression testing—with a Decision Engine, a reasoning model, retrieval-based repair memory, and an independent verification pipeline.

The central design principle is that the reasoning model should not be trusted to perform every security task. Instead, specialized tools generate objective evidence, the Decision Engine decides what additional work is necessary, the reasoning component interprets the evidence and proposes a repair when needed, and independent verification determines whether the repair should be accepted.

The system is intended to optimize performance, speed, precision, functionality, scalability, reliability, auditability, and efficient model usage. Python is proposed for orchestration, AI/model integration, retrieval, APIs, and workflow management, while Rust is proposed for performance-critical analysis, repository processing, coverage handling, and high-concurrency infrastructure.

2. Problem Statement

Modern software security tooling can identify suspicious code, crashes, dependency problems, or known vulnerability patterns, but discovery, root-cause analysis, repair, and verification are often separate activities. A generated patch may compile while failing to remove the vulnerability, breaking intended functionality, or introducing a new defect.

The proposed problem is therefore not simply 'detect vulnerabilities with AI.' It is to construct an autonomous system that can connect the entire security-repair lifecycle:

- Discover a potential vulnerability.
- Collect independent evidence and determine whether it is real.
- Understand the root cause and affected execution/data-flow paths.
- Retrieve relevant historical fixes, project conventions, and previous repair outcomes.
- Choose an appropriate repair strategy.
- Generate a minimal, project-consistent patch.
- Generate security tests that target the discovered weakness.
- Verify the patch through compilation, regression testing, security tests, re-analysis, and re-fuzzing.
- Learn from successful and failed repair attempts.
- Produce an auditable evidence chain for the final repair decision.

3. Core Objectives

Objective	Goal	Primary Mechanisms
Performance	Efficient execution under heavy analysis workloads	Rust workers, caching, parallelism
Speed	Reduce unnecessary expensive analysis	Change detection, risk prioritization, fast paths

Precision	Reduce false positives and unsafe repairs	Evidence fusion, reproduction, independent verification
Functionality	Support diverse repositories and workflows	Plugin-oriented analyzers, APIs, CI/IDE integration
Scalability	Process many jobs concurrently	Queues, worker pools, horizontal scaling
Auditability	Explain why a repair was accepted	Evidence graph and Repair Assurance Report

4. Key Innovations Beyond the Basic Problem

Repair Memory: Stores successful and failed repairs, their evidence, regression outcomes, and final decisions.

Risk-Based Computation: Focuses expensive analysis on recently changed, reachable, exposed, or security-critical code.

Automatic Security-Test Generation: Creates tests specifically designed to reproduce the discovered vulnerability.

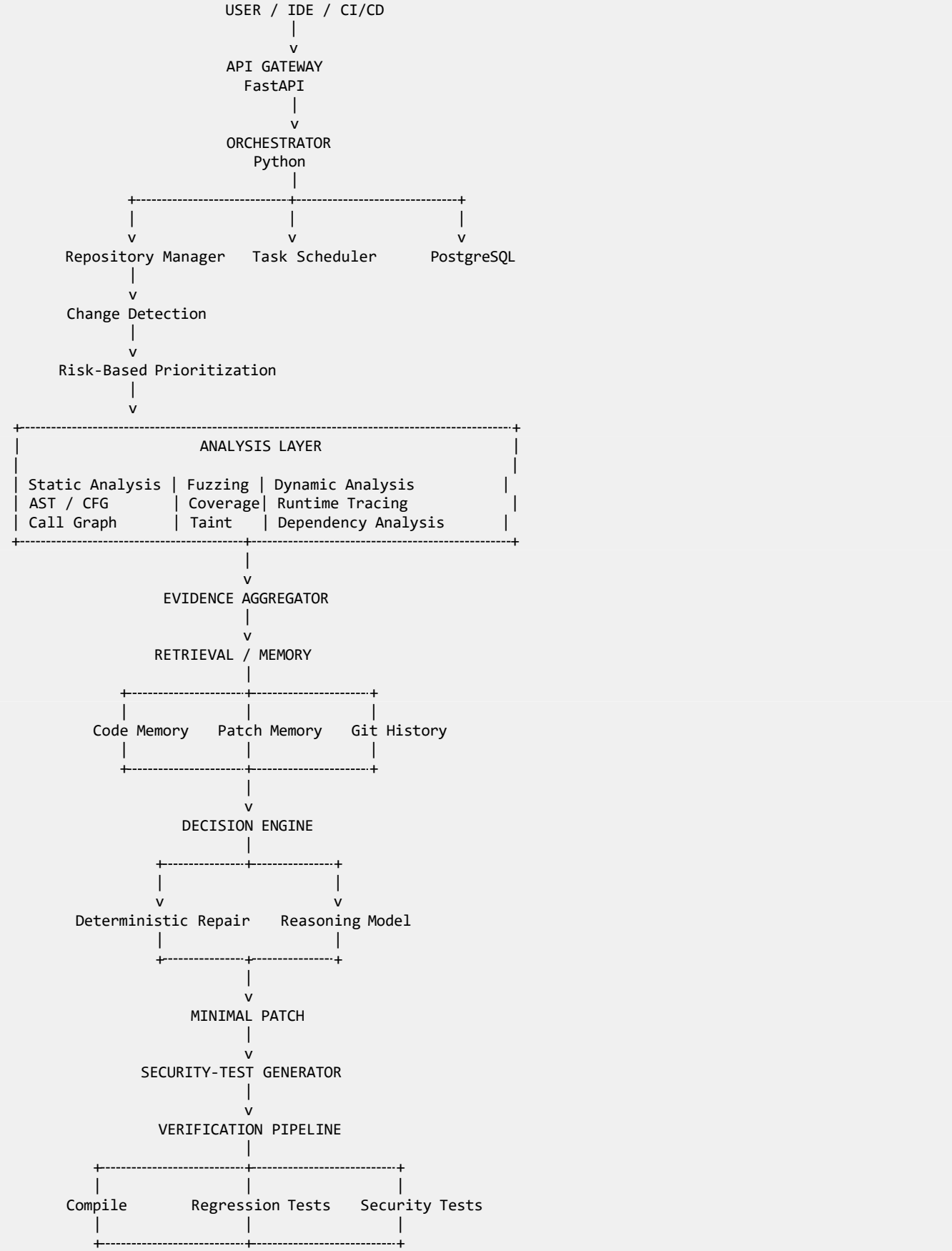
Minimal Patch Optimization: Prefers the smallest safe patch that satisfies security and regression requirements.

Failure Memory: Treats rejected repairs as useful knowledge rather than discarding them.

Patch Mutation Testing: Tests whether the generated security tests are strong enough to detect a reintroduced weakness.

Auditable Repair Proof: Maintains an evidence chain from discovery through final verification.

5. High-Level Architecture



6. Component Architecture

6.1 API Gateway

A FastAPI service provides the external interface for developers, CI/CD systems, IDE integrations, and future dashboards. Typical operations include starting scans, requesting analysis, retrieving job status, and obtaining repair reports.

6.2 Orchestrator

The Python orchestrator manages the end-to-end workflow. It does not perform every analysis task itself; instead it creates jobs, routes evidence, invokes the Decision Engine, coordinates retrieval and reasoning, and starts verification.

6.3 Repository Manager

Responsible for cloning repositories, checking out commits, detecting languages and build systems, identifying dependencies, preparing isolated environments, and exposing repository metadata to downstream components.

6.4 Change Detection

Compares revisions and identifies changed files, functions, classes, and potentially affected dependencies. This allows VAJRA to prioritize incremental analysis instead of repeatedly scanning an entire repository.

6.5 Risk Prioritization

Assigns risk scores using factors such as severity, exploitability, reachability, exposure, recent changes, complexity, historical vulnerabilities, dependency risk, and analyzer evidence.

6.6 Static Analysis Layer

Provides AST extraction, control-flow graphs, call graphs, data-flow and taint analysis, unsafe API detection, dependency analysis, and security-pattern detection. Mature external analyzers should be integrated rather than unnecessarily reimplemented.

6.7 Fuzzing Layer

Coordinates fuzzing engines such as AFL++, libFuzzer, or honggfuzz. It manages corpora, seeds, workers, crash reproduction, crash deduplication, coverage, timeouts, and prioritization.

6.8 Dynamic Analysis Layer

Executes targets in isolated environments and collects stack traces, runtime behavior, memory/sanitizer information, system calls, execution paths, and coverage.

6.9 Evidence Aggregator

Normalizes heterogeneous outputs into a common evidence representation so the Decision Engine and reasoning model can operate on structured, relevant evidence rather than raw logs.

6.10 Retrieval and Repair Memory

Provides semantic and structured retrieval over prior code, patches, failed repairs, Git history, vulnerability knowledge, project conventions, and verification outcomes.

6.11 Decision Engine

Determines whether a finding is sufficiently confirmed, whether more analysis is needed, whether a deterministic repair is available, whether model reasoning is required, which historical evidence is relevant, and which verification strategy should be applied.

6.12 Reasoning Model

Analyzes the selected evidence, identifies root cause, proposes a repair strategy, generates a patch, identifies required tests, and returns structured reasoning metadata. It should be replaceable and should not be treated as the source of truth.

6.13 Patch Generator and Minimality Evaluator

Produces project-consistent patches and compares candidates for size, affected files, complexity, API changes, dependency changes, and regression risk.

6.14 Security-Test Generator

Creates targeted security regression tests intended to reproduce the discovered vulnerability before the patch and demonstrate that the vulnerability no longer reproduces after the patch.

6.15 Verification Engine

Runs compilation, unit/regression tests, generated security tests, static re-analysis, dynamic analysis, re-fuzzing, and coverage comparison. A patch is accepted only when required verification criteria pass.

6.16 Failure and Repair Memory

Records rejected patches, failure reasons, regressions, security-test failures, and successful repair histories so later repairs can learn from previous experience.

6.17 Repair Assurance Reporter

Creates an auditable report containing the vulnerability, evidence, root cause, historical context, patch, tests, verification results, and final confidence/assurance information.

7. Evidence Model

A core architectural requirement is a normalized evidence object. The model should receive a compact representation of the most relevant facts instead of an entire repository.

```
{
  "target": {
    "repository": "example",
    "commit": "abc123"
  },
  "location": {
    "file": "src/parser.cpp",
    "function": "parse_input",
    "line": 143
  },
  "vulnerability": {
    "type": "heap-buffer-overflow",
    "severity": "high"
  },
  "static_analysis": {
    "finding": "...",
    "taint_flow": "...",
    "call_graph": "..."
  },
  "dynamic_analysis": {
    "crash": true,
    "stack_trace": "..."
  },
  "fuzzing": {
    "reproduced": true,
    "coverage": 82.4
  },
  "history": {
    "similar_change_found": true
  }
}
```

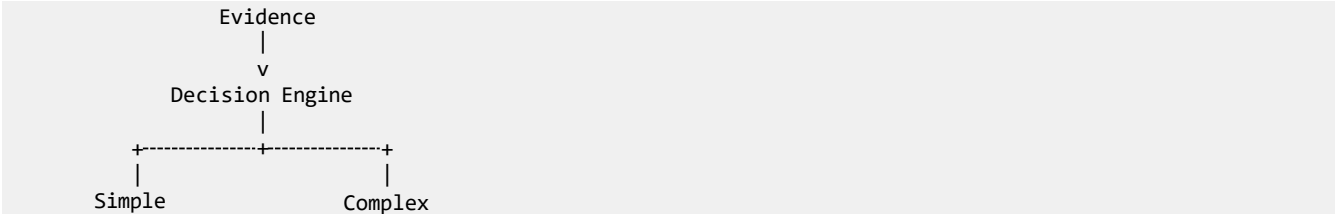
8. Retrieval / Memory Architecture

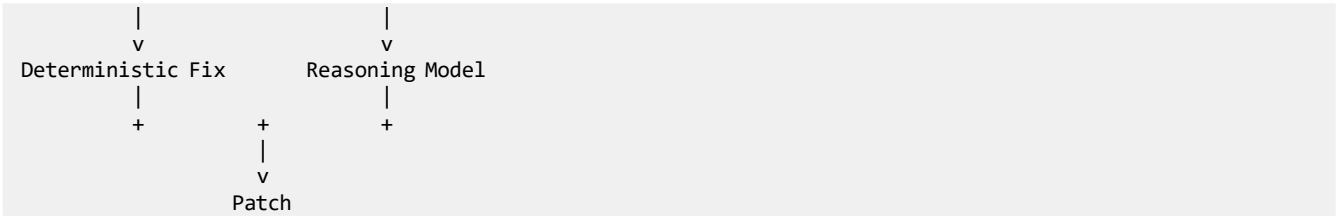
VAJRA should use a software-engineering memory rather than relying on conventional document-only RAG. The memory combines structured storage, semantic retrieval, and repository history.

- Code Memory — similar functions and implementations.
- Patch Memory — successful patches and their verification evidence.
- Failure Memory — rejected patches and why they failed.
- Git History — previous changes, commits, and context around modifications.
- Vulnerability Memory — vulnerability classes and remediation patterns.
- Project Memory — project-specific APIs, wrappers, conventions, and architectural constraints.
- Regression Memory — prior tests and outcomes associated with repairs.

The retrieval layer should answer questions such as: Has this vulnerability or similar code been fixed before? Why was a previous change made? Did a previous repair introduce a regression? Is there an existing project-specific security wrapper that should be reused?

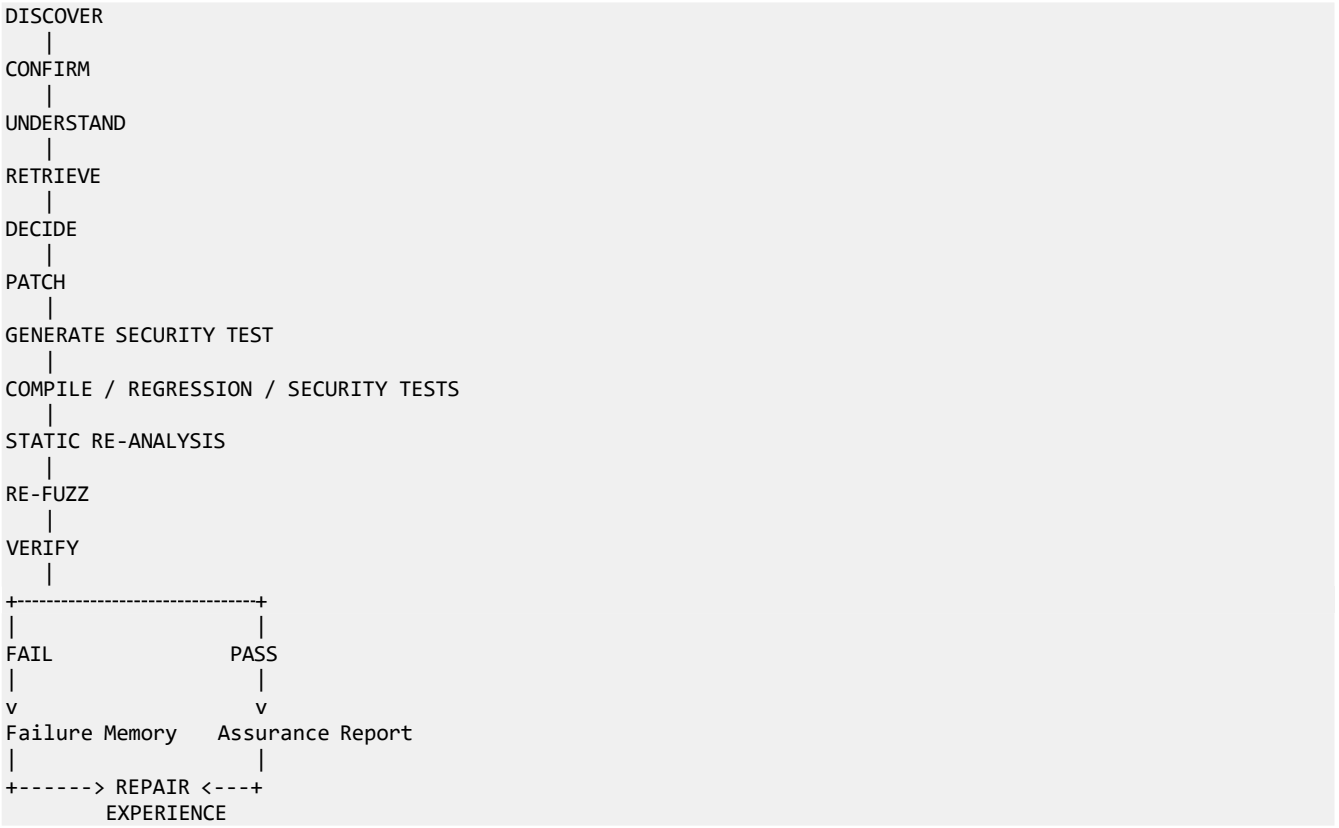
9. Decision Engine Logic





The Decision Engine is important for speed and reliability. Many findings can be handled with deterministic rules, known secure coding patterns, or project-specific wrappers. Complex or ambiguous cases can be routed to the reasoning model. This reduces unnecessary inference and makes the overall system more predictable.

10. Autonomous Repair and Verification Loop



Verification failures should return to the Decision Engine rather than simply terminating the job. The system can analyze the failure, store it in memory, and produce a new candidate patch. This creates an autonomous repair loop.

11. Auditable Repair Assurance

VAJRA should avoid claiming absolute proof of safety for arbitrary software. Instead, it should generate an evidence-based assurance that the identified vulnerability was mitigated and that no tested regressions were detected.

- Vulnerability and affected code.
- Original reproduction evidence.
- Root-cause analysis.

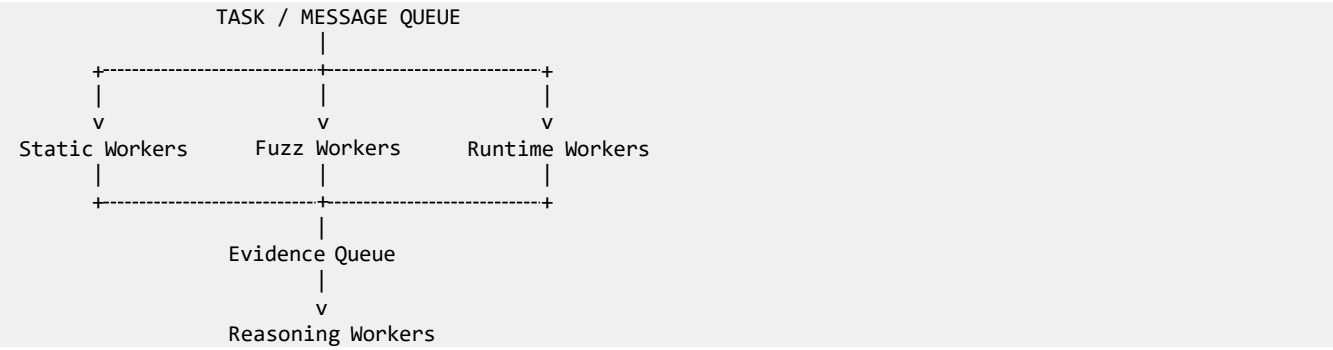
- Relevant historical retrieval results.
- Repair decision and strategy.
- Patch and changed files.
- Generated security test.
- Compilation and regression results.
- Static re-analysis results.
- Dynamic analysis results.
- Re-fuzzing results.
- Coverage comparison.
- Patch minimality metrics.
- Final verification decision.
- Confidence and limitations.

12. Technology Stack

Layer	Recommended Technology	Purpose
Orchestration	Python	Workflow, Decision Engine, AI integration
API	FastAPI / Python	External and service interfaces
Performance-Critical Services	Rust	Parsing, indexing, concurrency, analysis support
Relational Database	PostgreSQL	Jobs, vulnerabilities, patches, verification metadata
Vector Retrieval	Qdrant or equivalent	Semantic retrieval over code, patches, and knowledge
Queue / Messaging	Redis/RQ, Celery, RabbitMQ, or equivalent	Asynchronous job scheduling
Isolation	Docker / sandboxing	Safe execution of target software
Fuzzing	AFL++, libFuzzer, honggfuzz	Automated input generation and crash discovery
Frontend (future)	React / Next.js	Dashboard and visualization

13. Scalability Architecture

The architecture should be horizontally scalable. Long-running workloads are represented as jobs and distributed to worker pools. Static analysis, fuzzing, dynamic analysis, reasoning, and verification workers can scale independently.



```
                    |
                    v
          Verification Queue
                    |
                    v
        Verification Workers
```

14. Performance and Speed Strategy

- Incremental, change-aware analysis.
- Risk-based scheduling of expensive operations.
- Parallel static, fuzzing, and dynamic workloads.
- Caching of repository and analysis artifacts.
- Crash deduplication.
- Selective and targeted fuzzing.
- Compact evidence representations.
- Retrieval instead of unnecessarily large model contexts.
- A fast path for simple vulnerabilities.
- A deep path for ambiguous or high-impact vulnerabilities.
- Independent scaling of worker pools.

15. Precision and Reliability Strategy

Precision should come from independent evidence sources rather than model confidence alone. For example, a high-confidence repair might have a static finding, a reproducible fuzzing crash, dynamic confirmation, relevant historical evidence, a passing generated security test, passing regression tests, successful re-fuzzing, and removal of the original static finding.

16. Security and Isolation Requirements

- Treat repository source code, build artifacts, binaries, and generated inputs as potentially untrusted.
- Execute targets inside isolated containers or stronger sandbox environments.
- Restrict network and filesystem access according to the analysis policy.
- Separate the orchestration host from arbitrary target execution.
- Record tool versions and environment metadata for reproducibility.
- Keep analysis artifacts immutable once used in an assurance report.
- Make verification decisions traceable to specific tool outputs.

17. Development Roadmap

Phase 1 — Foundation: Repository Manager, FastAPI gateway, PostgreSQL, job system, Python orchestrator, Rust service framework, container isolation.

Phase 2 — Analysis: Integrate static analysis, AST/call graph extraction, dependency analysis, fuzzing, dynamic execution, and coverage.

Phase 3 — Evidence s Memory: Evidence Aggregator, vector retrieval, Git history indexing, patch memory, failure memory.

Phase 4 — Decision Engine: Risk scoring, prioritization, deterministic repair routing, model routing, verification strategy selection.

Phase 5 — Reasoning s Repair: Root-cause reasoning, patch generation, minimality evaluation, historical repair retrieval.

Phase 6 — Autonomous Verification: Security-test generation, build/test pipeline, static re-analysis, dynamic analysis, re-fuzzing, coverage comparison.

Phase 7 — Learning: Successful/failed repair learning, project-specific knowledge, outcome tracking.

Phase 8 — Distributed Scaling: Worker pools, queues, distributed fuzzing and verification, resource scheduling, monitoring.

18. Evaluation Plan

A research implementation should be evaluated against representative vulnerable programs, open-source repositories, synthetic vulnerability suites, and controlled regression scenarios. The evaluation should compare both security outcomes and systems performance.

- Vulnerability discovery rate and confirmed-vulnerability rate.
- False-positive rate.
- Vulnerability reproduction rate.
- Patch acceptance rate.
- Patch correctness and regression rate.
- Security-test effectiveness.
- Re-fuzzing outcomes.
- Patch size and changed-file count.
- Time to first confirmed finding.
- Time to verified repair.
- CPU and memory consumption.
- Reasoning-model inference count and latency.
- Cache/retrieval hit rate.
- Worker throughput.
- Scalability under concurrent repositories/jobs.
- Auditability and reproducibility of final reports.

19. Research Questions

1. How much can external security evidence reduce the reasoning workload required from a small or compressed model?
2. Does retrieval of previous successful and failed repairs improve patch correctness?
3. Can change-aware risk prioritization reduce analysis time without significantly reducing vulnerability discovery?

4. Do automatically generated security tests improve confidence in repair correctness?
5. Does patch minimality reduce regression risk?
6. Can independent evidence fusion reduce false positives compared with a single analyzer?
7. How does the system scale as the number of repositories and concurrent analysis jobs increases?
8. Can the Decision Engine reduce model inference cost while maintaining repair quality?

20. Expected Outcome

The intended outcome is not merely an AI vulnerability scanner or an automated code-generation assistant. VAJRA is intended to be an evidence-driven autonomous security engineering platform. Its distinguishing property is the closed loop connecting discovery, confirmation, historical retrieval, decision-making, minimal repair, security-test generation, independent verification, and learning.

The system should ultimately be capable of taking a repository or CI/CD change as input and returning either a verified minimal repair with a complete Repair Assurance Report or a structured explanation of why the system could not safely repair the issue.

21. Proposed One-Paragraph Definition

VAJRA is an evidence-driven autonomous cyber-reasoning and software repair system that combines static and dynamic analysis, fuzzing, dependency analysis, and regression testing to discover and confirm vulnerabilities. A Decision Engine combines this evidence with a memory of previous successful and failed repairs, prioritizes high-risk code, and coordinates deterministic or model-assisted patch generation. VAJRA generates security tests, produces minimal patches, and independently verifies them through compilation, regression testing, re-analysis, and re-fuzzing. Every repair produces an auditable evidence chain explaining the root cause, repair process, verification results, and confidence. Its modular distributed architecture uses Python for orchestration and reasoning and Rust for performance-critical analysis, enabling high performance, speed, precision, functionality, and scalability while allowing the reasoning model to remain small or compressed.

22. Important Technical Position

The system should be presented as providing evidence-based assurance rather than absolute proof of software safety. No finite testing and analysis pipeline can establish that arbitrary software contains no undiscovered vulnerabilities. The strength of VAJRA is instead that each accepted repair is supported by multiple independent, reproducible verification signals and a complete audit trail.